Enterprise SIEM Lab

Lightweight Security Log Analysis Prototype

Project type: Beginner cybersecurity / security monitoring prototype

Status: Prototype executed against bundled synthetic sample data. This report does not claim a live Wazuh deployment.

1. Abstract

Security Information and Event Management (SIEM) systems help security teams centralize and review security events. This project demonstrates a small part of that workflow: reading structured authentication events, counting failed login attempts by source IP, and creating a JSON alert when a threshold is reached. The implementation uses Python's standard library and runs offline on a modest Windows computer.

2. Problem Statement

Reviewing many authentication events manually can make repeated failed login activity harder to notice. A simple, transparent rule can help flag patterns for further investigation. A failed-login alert is only a signal; it does not by itself prove an attack.

3. Objectives

• Read structured, line-based authentication events from a local file.

• Count event types and skip malformed event lines safely.

• Detect repeated failed login attempts from a single source IP.

• Export a machine-readable JSON summary and document limitations.

4. Scope and Architecture

Input: synthetic sample_auth.log → Parser: main.py → Rule engine: AUTH-001 threshold check → Output: output/alerts.json. This is an offline demonstration. It does not collect live endpoint logs, connect to Wazuh, block traffic, or automatically respond to incidents.

Component	Purpose
Sample log file	Synthetic authentication events for repeatable testing
Parser	Reads timestamp, host, event and optional fields
AUTH-001 rule	Alerts if one source IP has at least 3 failed login events
JSON output	Stores counts, alert details and known limitations

5. Tools and Requirements

Language: Python 3.10+ recommended. Operating system: Windows 10 or another OS with Python installed. Dependencies: Python standard library only; no third-party package installation is required. The code and sample data need only a small amount of disk space.

6. Detection Rule

Rule ID AUTH-001: if the number of login_failed events for one source IP is greater than or equal to the configured threshold (default 3), create a medium-severity alert. The prototype counts all matching events in the input file and does not apply a rolling time window.

7. Test Procedure

From the extracted project folder, run: py main.py --input data/sample_auth.log --output output/alerts.json. If the py launcher is unavailable, try python instead. The bundled sample file contains synthetic events, including four failed logins from the reserved documentation IP 203.0.113.10, two successful logins and one unrelated service event, plus one malformed line.

8. Test Results

Metric	Observed result
Valid events processed	7
Malformed lines skipped	1
Alerts generated	1
Alert rule	AUTH-001
Alert source IP	203.0.113.10

These results were reproduced by executing the included code against the included sample file in the generation environment. The sample data is synthetic and does not represent a real incident.

9. Limitations

• No live collection of Windows Event Logs or network traffic.

• No Wazuh server, dashboard, indexer, or agent integration.

• No time-window correlation, user baselining, or false-positive suppression.

• No automated blocking, endpoint isolation, or production response.

• Detection depends on the input event format and configured threshold.

10. Future Enhancements

Possible future work includes unit tests, rolling time-window detection, Windows Event Log ingestion in an authorized lab, a simple dashboard, richer alert context, and integration with a dedicated SIEM after checking hardware resources and deployment requirements.

11. Conclusion

The prototype demonstrates a basic SIEM-style workflow: parse events, apply a detection rule, and generate a structured alert. It is suitable as a small educational code submission, provided it is described accurately as a prototype rather than a production SIEM deployment.

Appendix: Project Files

main.py — analysis code; data/sample_auth.log — synthetic events; requirements.txt — dependency note; README.md — setup and usage instructions; output/alerts.json — generated output after running the program.